



User manual

How to use Memora, what happens to a memory once you send it, exactly what is stored – and an honest list of what has not been built yet.

VERSION 0.1 · 21 AUGUST 2026 · EVERY EXAMPLE BELOW IS REAL OUTPUT FROM A RUNNING INSTANCE

CONTENTS

1. **The idea in one page** — what Memora replaces
2. **Getting set up** — keys, environments, install
3. **The two calls** — remember and recall
4. **What happens when you store a memory** — the four stages, timed
5. **What is actually stored** — every table, with real rows
6. **What happens when you ask for a memory** — retrieval and ranking
7. **Reading a result** — what each field means and when to trust it
8. **The dashboard**
9. **Using it from an agent** — MCP
10. **Recipes** — the patterns worth copying
11. **Limits, errors and what to do about them**
12. **What we have not built yet** — the missing links

The idea in one page

An AI application has no memory between sessions. The usual workarounds are to resend the whole conversation every time, or to embed the transcript into a vector store and hope similarity finds the right bit later. Both store **text**. Neither can tell you what is currently true about a person.

Memora stores **knowledge** instead. You send it what someone said; it works out which parts are durable, keeps those, and throws away the rest. When something changes, it links the new version to the old one and records why. When two things cannot both be true, it says so instead of picking a side. Later, you ask a question and get back the handful of facts that bear on it — each one carrying its own explanation of why it was returned.

The mental model: Memora is not a database you write rows to. It is closer to an assistant who listens to your conversations, keeps notes on what matters, notices when a note goes out of date, and hands you the relevant page when you need it. You do not tell it what to remember. You tell it what happened.

What it is good at

Facts that persist	Preferences, constraints, goals, decisions, relationships — the things a person would be annoyed to repeat.
Facts that change	A job, a city, a preference that reversed. The old value stays readable; the new one supersedes it with a reason.
Facts that conflict	A stated allergy and a later contradicting statement. Both are flagged rather than one silently winning.
Knowing why	Every memory keeps the exact text it came from, and every change keeps the reasoning behind it.

What it is not

Not a transcript store	It deliberately discards most of what you send. If you need the raw conversation, keep your own copy — Memora keeps a short source excerpt for audit, not the full history.
Not a general vector database	You cannot store arbitrary documents in it. It stores statements about people.
Not a cache	Writes cost a model call. Do not call <code>remember</code> on every keystroke — call it on a completed turn.

Getting set up

01 Create an organisation, a project and an environment

Sign up in the dashboard. An **organisation** is your company. A **project** is usually one application. An **environment** is **development** or **production** within that project – and this is the important one, because memories in different environments never mix.

02 Generate an API key

Keys belong to one environment. A development key physically cannot read production memories – it is not a filter, it is a different scope. Keys also carry scopes: **read**, **write**, or both. Give your backend a read+write key and anything analytical a read-only one.

```
export MEMORA_API_KEY=sk_live_...
```

03 Install the client

```
npm install @memora/client
```

```
import { Memora } from "@memora/client";

const memora = new Memora({ apiKey: process.env.MEMORA_API_KEY! });
```

There are no dependencies and it runs anywhere **fetch** does – Node, Bun, Deno, Workers. An environment-scoped key belongs on a server; never ship one to a browser.

Choosing a user id. Every memory is filed under a **user_id** that you choose — your own identifier for that person. It is opaque to Memora. Use a stable internal id, not an email address that might change, and never a value that itself contains personal data you would not want stored.

The two calls

Everything else in this manual is detail. The whole integration is two calls, in this order, around whatever your application already does.

```
// 1. BEFORE you answer – pull what is known that bears on this message
const { results } = await memora.recall({
  userId,
  query: userMessage,
  topK: 5,
});

const whatWeKnow = results.map(m => `- ${m.content}`).join("\n");
// ...put whatWeKnow in your system prompt, then call your model as usual

// 2. AFTER the turn – hand Memora what was said
await memora.remember({ userId, content: userMessage });
```

Recall before, remember after. That order matters: recalling first means the current turn is answered with everything known up to now, and remembering after means this turn is available for the next one. Reversing them wastes a model call and changes nothing.

What you get back from a write

```
{
  outcomes: [
    { candidateContent: "Is vegan.",
      decision: "ADD",
      memoryId: "a3f1...",
      reconciliation: "pending" },
    { candidateContent: "Has a serious peanut allergy.",
      decision: "ADD",
      memoryId: "b7c2...",
      reconciliation: "pending" }
  ]
}
```

One sentence became two memories, and a third of the message – the greeting – became nothing, because it was not worth keeping. **"pending"** means the memories are stored and readable now, and are being judged against what was already known. That judgement takes a few seconds and does not block you.

What happens when you store a memory

Four stages. The first three happen before your call returns; the fourth happens after, which is why writes are fast despite doing a lot of thinking.

01 Extract — one model call

Your text goes to Claude, which pulls out durable statements about the user and discards reactions, acknowledgements and small talk. Each candidate gets a type, a confidence and an importance. Relative dates are resolved against any timestamp in the text, so "yesterday" becomes a real date.

```
// you sent:
"Hey - I'm Priya. I'm vegan, and I'm allergic to peanuts, it's serious."

// extraction produced:
[fact]      The user's name is Priya.      confidence 0.99
[preference] Is vegan.                    confidence 0.98
[fact]      Has a serious peanut allergy.   confidence 0.99
```

02 Embed — one batched call

All candidates are turned into 1024-dimension vectors in a single request, no matter how many there are. This is what makes semantic search possible later.

03 Store and return

Every candidate is written, each with a row recording the exact source text it came from, and a job is queued to judge it. Your call returns here – measured at **5.5 to 8.3 seconds** in the example run, almost all of it the extraction call.

04 Reconcile — after the response

Now the interesting part. Each new memory is compared against the closest existing ones and Claude decides what it actually is:

VERDICT	WHAT HAPPENS
ADD	Genuinely new. Nothing to repair.
UPDATE	A newer version of an existing fact. The new row links to the old one, the old is marked <code>superseded</code> , and the reason is recorded.
MERGE	Extra detail on something known. The two are combined into one fact.
IGNORE	A restatement. The duplicate is archived and the original's freshness is refreshed — repetition is treated as confirmation.
FLAG	Cannot both be true. Both memories are marked <code>flagged</code> and a contradiction record is written.

Why afterwards and not during? Because judgement is the slow part, and nothing about your user's experience improves by making them wait for it. The queue is a real database table, so a crash, a deploy or a failed model call means the job is retried — never silently skipped.

What is actually stored

Yes — the storage layer is built and running. It is PostgreSQL with the `pgvector` extension, and everything lives in one database: the memories, their embeddings, the full-text index, the job queue, the rate limit counters. There is no second datastore to keep in sync.

Here is the actual table after the three messages in Section 4, taken from a live instance:

content	type	status	conf	ver	judged
Has a serious peanut allergy.	fact	active	0.99	-	yes
The user's name is Priya.	fact	active	0.99	-	yes
Is vegan.	preference	active	0.98	-	yes
Works at Cobalt Labs	fact	superseded	0.90	-	yes
Started a new job last month	event	active	0.85	v2	yes
Moved from Berlin to Lisbon last month	event	active	0.90	-	yes
Currently lives in Lisbon	fact	active	0.90	-	yes
Previously lived in Berlin	fact	archived	0.85	-	yes
Wants the assistant to remember their pea..	instruction	archived	0.95	-	yes
Cannot eat peanuts (peanut allergy or int..	fact	archived	0.98	-	yes

Ten memories from three sentences. Note what reconciliation did without being asked: the third message repeated the peanut allergy, so those two candidates were **archived** as restatements rather than stored twice — and the original was refreshed instead.

The tables

TABLE	WHAT IT HOLDS
<code>memories</code>	The statement itself, its type, status, confidence, importance, a 1024-dimension embedding, a full-text index, a link to the version it replaced, and the ids of the end user, agent and session it belongs to.
<code>memory_evidence</code>	Why this memory exists. One row per event, holding the source excerpt, the source type and id you passed in, and Claude's stated reasoning for any change.
<code>contradictions</code>	Pairs of memories that cannot both be true, with the explanation and a resolution once a human settles it.
<code>reconciliation_jobs</code>	The judgement queue — status, attempts, last error. This is what makes the audit trail survive a crash.
<code>experiences</code>	A separate log for what an <i>agent</i> learned attempting a task, rather than what is true about a person.
<code>usage_events</code> , <code>api_requests</code>	Every model call and every billable request, per project and per key.

An evidence trail, in full

This is every row recorded for the peanut allergy — the answer to "why does it believe this?":

event	source	excerpt	reasoning
extracted	chat msg_1	Has a serious peanut allergy.	-
reconfirmed	chat msg_3	Cannot eat peanuts (peanut allergy)	The candidate restates the
			existing memory "Has a serious..."

```
reconfirmed | chat msg_3 | Wants the assistant to remember... | This is essentially a  
restatement | | | of the existing peanut allergy...
```

The **source** column is whatever you passed as **source** on the write. Pass your own message ids there and you can walk from any memory back to the exact message in your system that produced it.

What happens when you ask for a memory

Recall runs two searches and fuses them. Embeddings find what a question is **about**; a full-text index finds the exact strings embeddings blur — names, ticket numbers, cluster ids. A memory found by either route makes the candidate list; one found by both ranks higher.

The candidates are then scored on three things, not one:

SIGNAL	WEIGHT	MEANING
Match	60%	How well it matches the question, semantically and by keyword.
Confidence	25%	How sure we are the fact is true, based on how it was stated and how often confirmed.
Freshness	15%	How recently it was confirmed. Decays smoothly; halves at about 62 days.

Real output from the instance above — note that the food question surfaces the dietary facts without either word appearing in the query:

Q: "can they eat the pad thai?"

0.69	Is vegan.	preference	• matched on meaning
0.69	Has a serious peanut allergy.	fact	• matched on meaning
0.68	The user's name is Priya.	fact	• matched on meaning

Q: "where do they live?"

0.97	Currently lives in Lisbon	fact	• matched on BOTH
0.69	Is vegan.	preference	• matched on meaning
0.68	Has a serious peanut allergy.	fact	• matched on meaning

Read those numbers honestly. The right answers came first both times, and the keyword hit on "Lisbon" scored 0.97 — the fusion working exactly as designed. But look at the food question: the correct memories scored 0.69 and an irrelevant one scored 0.68. The ranking is right, the *margin* is almost nothing.

The cause is that confidence and freshness are near-identical across fresh memories, so they add a constant to everything and compress the differences. It works at ten memories. At two hundred — which one dense conversation produced in our benchmark — it is why the right fact sometimes does not make the top five. This is the single most valuable thing to fix next, and it is a ranking problem, not an intelligence one.

Reading a result

```
{
  memoryId:      "b7c2-...",
  content:       "Has a serious peanut allergy.",
  type:         "fact",
  status:        "active",
  relevanceScore: 0.69,
  similarity:    0.32,
  confidence:    0.99,
  freshness:     1.00,
  matchedOn:     "meaning",
  reason:        "weak semantic match (0.32), high confidence, recently confirmed",
  history:       []
}
```

FIELD	WHAT TO DO WITH IT
content	Put this in your prompt. It is written as a third-person statement so it reads correctly there.
status	Check this. <code>active</code> is a settled fact. <code>flagged</code> means it contradicts another memory and neither is settled — show both, or ask the user, but never state it as fact.
relevanceScore	The ranking number. Compare within one result set; do not compare across queries.
confidence	How much to trust the fact itself. Use <code>minConfidence</code> on the call if you only want near-certainties.
freshness	1.0 is confirmed just now. Below 0.3 the system is telling you this has not been mentioned in a long time.
matchedOn	<code>keyword</code> means an exact string matched — usually a very literal hit. <code>both</code> is the strongest signal.
reason	Plain English. Log it, or show it in your own admin UI when someone asks why the assistant said something.
history	Earlier versions of this same fact, newest first. Empty for a fact that never changed. This is what answers "where did they used to work?".

A pattern worth copying. When rendering memories into a system prompt, mark the flagged ones:

```
results.map(m => m.status === "flagged"
  ? ` - ${m.content} (CONFLICTS with another memory - ask, do not assume)`
  : ` - ${m.content}`
)
```

Asking why, in full

```
await memora.explain(memoryId);
// → the memory, every evidence row, every version with the reason it changed,
//    any contradictions, and whether it has been reconciled yet
```

The dashboard

Everything above is visible without writing code. The dashboard lets you browse and filter memories by user, type and status; open any one to see its evidence trail and version timeline; edit or archive one by hand; and use a playground to try `remember` and `recall` against a real environment. Memories still waiting to be judged show a "reconciling" chip, and ones whose judgement failed show as "unreconciled" rather than pretending to be settled.

Using it from an agent

An MCP server ships alongside the API, so Claude Code, Cursor or any MCP client can use Memora directly with no integration code:

```
claude mcp add memora --env MEMORA_API_KEY=sk_live_... -- npx -y @memora/mcp
```

The agent gets four tools: `remember`, `recall`, `record_experience` – what it learned attempting a task, especially a failure – and `recommend`, which tells it what previous attempts suggest before it starts. Flagged memories are surfaced to the agent as explicit conflicts, so it asks rather than assuming.

Recipes

A chat application

```

async function handleTurn(userId, message) {
  const { results } = await memora.recall({ userId, query: message, topK: 5 });

  const reply = await yourModel({
    system: `What you know about this user:\n` +
      results.map(m => `- ${m.content}`).join("\n"),
    message,
  });

  // fire and forget – do not make the user wait for the write
  void memora.remember({ userId, content: message, sessionId: conversationId });
  return reply;
}

```

A support agent that must not guess

```

const { results } = await memora.recall({
  userId, query: ticketSubject, minConfidence: 0.8, types: ["fact", "instruction"],
});

const unresolved = results.filter(m => m.status === "flagged");
if (unresolved.length) escalateToHuman(unresolved);

```

Several agents, one user

```

await memora.remember({ userId, content, agentId: "billing-bot" });
await memora.recall({ userId, query, agentId: "billing-bot" });
// the sales bot never reads the billing bot's notes

```

Safe retries

```

await memora.remember({ userId, content, idempotencyKey: messageId });
// the same message id twice returns the first result – not a second set of memories

```

Limits, errors and what to do about them

STATUS	MEANS	WHAT TO DO
401	Key missing, wrong or revoked.	Do not retry. Check the key.
403	The key lacks the scope — usually a read-only key trying to write.	Do not retry. Use a write key.
409	An idempotency key was reused with different content.	A bug on your side. Use a new key per distinct write.
429	Too many requests this minute.	Retry after the <code>Retry-After</code> header. The SDK does this for you.
402	Monthly quota spent.	Do not retry — waiting will not help. Upgrade the plan.

Every response carries `RateLimit-Limit` and `RateLimit-Remaining` , so you can back off before being refused rather than after.

PLAN	WRITES / MONTH	READS / MONTH	REQUESTS / MINUTE
Free	1,000	10,000	60
Starter	25,000	250,000	300
Pro	250,000	2,500,000	1,200
Enterprise	unlimited	unlimited	6,000

On latency. A write costs a model call — expect seconds, not milliseconds, and never put it on the path of rendering a response to your user. A read is fast in principle, but the numbers you would measure today are dominated by an embeddings rate limit on our side, described overleaf.

What we have not built yet

This is the honest list. Everything above works today and every example in this manual is real output. Everything below is required before Memora is a product someone can rely on rather than a system that works.

Blocking — nothing runs for real until these are done

MISSING	WHY IT MATTERS
BLOCKER Embeddings capacity	The embeddings account has no payment method, which caps it at three requests per minute . That is why the second recall in Section 6 took twenty seconds and the first took half a second. No latency figure in this manual should be trusted until this is lifted, and no real traffic can be served.
BLOCKER Hosting	The container is built and boot-tested, the deploy runbook is written, and migrations run automatically before the app starts. There is no server, domain, TLS certificate or managed database to put it on. Today it runs on a laptop.
BLOCKER Billing	Usage is metered per model call and per request, and plan quotas are enforced. Nothing charges anyone, and there is no upgrade path — so the plan table opposite is real enforcement against prices nobody can pay.
BLOCKER Backups	No automated backup, and no restore has ever been tested. An untested backup is a belief, not a backup.

Serious — a buyer will ask for these

MISSING	WHY IT MATTERS
SERIOUS Team accounts	One owner per organisation. No invites, no roles, no password reset, no email verification. A team cannot share an account safely.
SERIOUS Alerting	Failures land in container logs. Nothing pages anyone when reconciliation starts failing, which is exactly when the audit trail silently stops being complete.
SERIOUS Legal and privacy	Memora stores personal data about your users. Terms of service, a privacy policy, a DPA, PII redaction at write time and a dashboard audit log are prerequisites for a real customer, not polish.
SERIOUS Event timestamps	A memory is stamped with when it was <i>written</i> , not when the thing happened. Replaying a historical conversation dates everything to today. Extraction works around this by reading dates out of the text; a first-class <code>occurred_at</code> field is the real fix.
SERIOUS Choice of models	Claude and Voyage are wired in directly. An enterprise on Bedrock or Azure cannot adopt Memora without a bring-your-own-model path.

Known quality gaps

GAP	DETAIL
NEXT Ranking at scale	Described in Section 6. One dense conversation produced 210 memories for a single user, and three of five benchmark misses were the right fact never reaching the top five. Confidence and freshness need to discriminate rather than offset.
NEXT Temporal questions	50% on the temporal category of the LoCoMo benchmark — our weakest area, and the same root cause as event timestamps above.
NEXT Benchmark coverage	Scored 75% on a twenty-question sample of LoCoMo. The full benchmark is 1,986 questions and has not been run, so the number is indicative rather than publishable.

WATCH Occasional malformed model output

Twice during development, a model call returned structurally corrupt output — once a mangled field name, once a memory whose content was the single word "test". Both are guarded against now (bad decisions are retried, bad grades are re-asked) but the underlying flakiness is real and worth watching in production.

LATER Platform surface

No batch writes, no export, no webhooks. Individually small; collectively the difference between an API and a platform.

What this list is not. None of it is research, and none of it is unknown. The engine — extraction, reconciliation, versioning, contradiction detection, hybrid retrieval — is built and measured. What remains is the ordinary work of turning a system that works into a service someone else can depend on.